

OntoKG: 用内在-关系路由做本体导向的知识图谱构建

OntoKG: Ontology-Oriented Knowledge Graph Construction with Intrinsic-Relational Routing

原文: Yitao Li、Zhanlin Liu、Anuranjan Pandey、Muni Srikanth (ProRata.ai),
arXiv:2604.02618, 2026 年 4 月。代码与 schema 开源在 github.com/Prorata-ai/OntoKG。

核心内容

Q: 这篇论文要解决的是什么问题? A: 把一个原始的大规模知识图谱整理成带类型的属性图时, 必须做几个结构性决定: 哪些东西成为节点、哪些属性变成边、由什么 schema 来管这些选择。现有做法把这些决定埋在流水线代码里 (YAGO、DBpedia), 或者临时抽关系 (各种基于大模型的流水线), 结果 schema 和构建过程绑死, 事后没法复用。作者主张 schema 应当从一开始就为下游的本体级任务设计, 而不是构图的副产品。

Q: 核心机制是什么? A: 内在-关系路由。每一个属性在 schema 设计时就被判成两类之一: 内在属性 (intrinsic) 变成节点上的字段, 供查询用, 例子是出生日期; 关系属性 (relational) 变成图上可遍历的边, 例子是雇主。判完之后路由到对应的 schema 模块。这一刀同时定下两件事: 节点范围 (哪些实体成为节点、挂哪些属性) 和边界 (哪些属性成为可遍历的边、哪些只是标量属性)。

Q: schema 具体长什么样? A: 一组声明式 YAML 文件, 每个大类一个文件, 同时充当分类规则和输出结构的唯一正本。结构是两层。第一层是大类 (category), 每个大类由一组「闸值」(gate values) 定义——某个实体的类型断言属性 (比如「是某类的实例」) 只要命中闸值集合里任何一个, 就归入这个大类。大类按固定优先级评估, 先命中者胜, 所以每个实体最多属于一个大类。第二层是模块 (module), 每个模块有三个字段: 类型 (内在还是关系)、指示器 (一组属性-取值条件, 命中就激活这个模块)、取值属性 (这个模块要抽取哪些属性)。

Q: schema 是一次设计好的吗? A: 不是, 是迭代出来的。从一个种子 schema 开始, 每一轮跑分类, 然后看两类失败: 没有大类命中的实体, 以及归了类但没有任何模块激活的实体。针对这两类失败去扩闸值、补模块。作者明确指出: 计算步骤 (分类、失败集划分、比率计算) 全是确定性且可并行的, 瓶颈只在三个需要语义判断的「决策 oracle」——判大类、判模块、判要不要新建或合并拆分模块。

Q: 那三个需要判断的地方谁来做? A: 一个用 Claude Opus 4.6 驱动的 agent workflow, 人做 review。作者说他们要防两件事: 大模型会凭一个名字编出一个看起来合理但不存在的实体编号; 以及只看名字判断不了一个陌生的实体类型该归哪一类。解法是给 agent 配五个「接地工具»: 标签查询 (把编号解析成规范英文名, 反向也行)、在线类型查询 (拿到某个类型的实例数、描述、样本实例)、schema 校验器 (查 YAML 结

构、阈值与模块的同步、编号格式、跨大类阈值互斥)、覆盖分析(出每个大类的覆盖统计、交叉表、以及哪些实体没分到模块)、未分类枢纽分析(找出被大量已分类实体引用、但自己还没分类的实体类型)。**agent 引用的每一个标识符都要过工具核实。**

Q: 结果如何? A: 在 2026 年 1 月的 Wikidata 快照(约 1 亿条目)上跑。先做基于规则的清洗, 识别出 3460 万实体的核心集——因为原始 1 亿里约三分之二是自动批量导入的东西(科学论文、基因这类)和基础设施条目(消歧页、分类页)。然后迭代路由, 最终 schema 有 8 个大类、94 个模块(56 个内在、38 个关系), 大类覆盖率 93.3%, 在已分类实体里模块分配率 98.0%(没模块的比例从 15.8% 降到 2.0%)。导出的属性图是 3400 万节点、6120 万条边、38 种关系类型。

Q: 为什么说这个 schema 是「一等公民」? A: 作者用五个下游应用来证明——每个应用都只消费 schema 或分类产物, 不碰原始数据和构建流水线。五个是: 本体结构分析与主题子图抽取、评测集标注审计、实体消歧、领域定制、以及用 schema 当 prompt 做大模型抽取。其中实体消歧在 BLINK 评测的受控候选子集上, 宏平均准确率 83.9%, 对照的 YAGO 4.5 是 81.5%, 高 2.4 个百分点。

背景补充 / 点评

先补三个理论来源, 这是这篇论文最容易被跳过、但对建模最有用的一节。

作者说内在与关系这个两分, 理论上来自三个传统。第一个是 Guizzardi 的统一基础本体(UFO), 它区分「刚性种类」(rigid sortal, 构成实体身份本质的类型)和「角色/混入」(role/mixin, 偶然的、依赖关系而存在的类型)。一个人是「人」这件事是刚性的, 是「雇员」这件事是角色。第二个是 Kiczales 等人在软件工程里提出的「横切关注点」(cross-cutting concerns)——那些无法干净地封装进主分解结构里某一个模块的东西, 日志和权限是经典例子。第三个是 Ranganathan 的分面分类法(faceted classification), 图书馆学的方法, 把主题拆成若干互相独立的「面」, 而不是硬塞进一棵单一的层级树。

作者把这三个传统落成一条可机械判断的启发式规则: **一个模块如果出现在多个大类里, 它就是关系模块。**宗教横跨 7 个大类, 奖项、教育、军事各横跨 6 个, 这些都是横切关注点; 而电影这个模块只出现在「创意作品与媒体」这一个大类里, 河流只出现在「地点」里, 这些是内在模块。作者自己说, 这条规则的用意是「降低传统基础本体所要求的哲学判断的程度」——把哲学问题换成一个能数出来的问题。

Wikidata、QID、PID 这几个词: Wikidata 是维基媒体基金会的协作式知识库, 每个实体有一个以 Q 开头的编号(QID), 每个属性有一个以 P 开头的编号(PID)。P31 是「是某类的实例」, P279 是「是某类的子类」, 这两个属性就是论文里做大类判断的类型断言属性。

我的判断: 这篇论文和我们的项目在结构上是同一个形状, 只是领域不同, 所以它的可借鉴密度是四篇里最高的。

它最深的洞察不是那个两分, 而是把「需要人判断的部分」和「可以确定性执行的部分」彻底切开。整条流水线只有三处需要语义判断, 其余全是确定性、可并行、可重跑的。这个切法带来两个后果: 一是人力只花在真正需要判断的地方; 二是每次改 schema 都能立刻 A/B 对比——他们的 Rust 分类器跑完 3460 万实体只

要五分钟（约每秒 11 万条）。第二点是前提性的：如果改一次 schema 要跑一天才知道结果，迭代收敛就不可能发生，人会退回一次性设计。

第二深的地方是收敛阈值的处理。他们把两个覆盖率的目标都设在 0.9 左右，然后明确说：剩下的长尾是稀有或含糊的类型，本来就不该属于任何一个大类，oracle **正确地拒绝了分配**，剩下的差距反映的是 schema 覆盖的真实边界，不是算法失败。这句话的分量在于它把「没分类」从一个要消灭的指标变成了一个合法状态。

第三处值得抄的是那个自文档化约定：YAML 里每一个 QID 和 PID 都必须带一条行内注释写上它的规范英文名，而且这条注释由校验器核对。这样 agent（或者人）能直接读 YAML 改定义，不需要逐个去查编号是什么意思。这是一条很小的规定，但它决定了本体文件能不能被人 and 模型直接读懂。

留白与要打折的地方：那个消歧的 +8.0 个百分点是在「候选超过 20 个」这一档拿到的，而那一档只有 25 条样本，作者自己也说了「效应量应当理解为方向性的」。所以真实的收益应该看宏平均那 2.4 个百分点，别被 8.0 那个数字带走。另外他们在结论里承认一个尚未解决的问题：现在这个 schema 是针对 Wikidata 精化出来的，一旦要接入外部来源（比如大模型抽出来的知识图谱），会冒出对不上现有模块清单的新关系类型，怎么扩展 schema 还没有方法。这一条恰好是我们要走的路——我们的输入就是大模型从对话里抽出来的东西，所以这个坑我们绕不开。

对我自己的启示，有五条，前三条能直接落地。

第一条，**内在与关系两分给了「画像和 memory 怎么分」一个能判定的判据**。学员画像和记忆的边界一直是按稳定性划的，而稳定性是连续量，划不出干净的线。按这篇的分法：岗位是内在属性，挂在学员节点上；「正在带一个新人」是关系，是学员和另一个人之间的边。这条线是离散的，能判。而且它顺带解决了存储问题——内在属性走表，关系走图，按访问模式分家。

第二条，**声明式文件是唯一正本，抽取 prompt 是编译产物**。这篇论文的 8 个大类映射成实体类型分类法，模块名映射成细粒度语义标签，prompt 模板直接从 schema 生成。这和本仓库现在做的事情完全同构：指标注册中心的 DSL 声明经编译器生成 SQL。做画像本体要走同一条路——本体是正本，抽取 prompt 是编译出来的，不是手写的。手写 prompt 就意味着同一份知识有两个正本，两边会各自漂移。

第三条，**产物里预生成自然语言句子这个设计要抄**。他们的分类产物是 Parquet 文件，每条声明除了带属性标签、取值标签、限定词，还带一个预先生成好的自然语言句子，句子里所有编号都已替换成英文名，比如「某人的职业是军官（军种：皇家海军）」。这个字段的用途是直接进 prompt 或者做文本检索，下游不用再解析编号。落到画像上：每条画像陈述除了结构化字段，同时存一句人话，供 Learning Lab 的对话直接引用。这解决了会上讨论「画像怎么被消费」时那个隐含的麻烦——结构化的东西塞进 prompt 需要拼装，拼装逻辑一散在各个消费方就漂移了。

第四条要警惕的：**他们花了整整一节做数据清洗，先划「哪些实体根本不该进图」，才开始分类**。1 亿条目里三分之二被清掉了。而且清洗规则里有一条「策展分数」专门保护知名实体不被误杀：一个实体如果有丰富的编辑信号（有图片、有坐标、有官网算强信号，各计 3 分；有奖项、有代表作算中等，计 2 分；有国别、有行业算弱信号，计 1 分），即使它的类型命中了批量导入的特征，也保留。对我们的对应问题是：从对话里抽

出来的候选里，有多大比例根本不该进画像（情境模拟里的假设、玩笑、口误）？这一步要在抽取之前定，不是抽完再筛。

第五条，**他们承认的那个未解决问题正好是我们的主路**。他们的 schema 是针对一个稳定的结构化数据源精化出来的，接入大模型抽取结果时会遇到「新关系对不上现有模块」的问题，作者列了三种应对方向：能映射到现有模块就映射，需要扩展就扩展模块，实在对不上就新建模块。这三种应对的判据他们没给。我们要做的正是这件事，而且没有现成答案可抄——这一处必须自己定判据，并且要有机制记录「一条新关系是被映射了、被扩展了、还是新建了」，否则本体会在无人察觉中膨胀。

文中金句

"The core mechanism is intrinsic-relational routing, which classifies every property as either intrinsic (a node attribute for lookup, e.g., birth date) or relational (a traversable graph edge, e.g., employer)." 核心机制是内在-关系路由：它把每一个属性判成内在的（供查询用的节点属性，例如出生日期）或关系的（可遍历的图边，例如雇主）。

"The computational steps—classification, failure partitioning, rate computation—are deterministic and parallelizable. The bottleneck lies in three decision oracles that require semantic judgment." 计算步骤（分类、失败集划分、比率计算）是确定性且可并行的。瓶颈在于三个需要语义判断的决策 oracle。

"The oracles correctly decline to assign them, and the remaining gap reflects genuine coverage limits of the schema rather than algorithmic failure." oracle 正确地拒绝为它们分配（类别），剩下的差距反映的是 schema 覆盖的真实边界，而不是算法的失败。

"Grounding tools verify each identifier used in schema decisions against the knowledge graph at decision time, addressing the hallucination problem at its source rather than through post-hoc validation." 接地工具在做决定的当时，就把 schema 决策中用到的每一个标识符对着知识图谱核实一遍，从源头上解决幻觉问题，而不是靠事后校验。

"Existing approaches embed these decisions in pipeline code or extract relations ad hoc, producing schemas that are tightly coupled to their construction process and difficult to reuse." 现有做法把这些决定嵌进流水线代码，或者临时抽取关系，产出的 schema 与其构建过程紧耦合，难以复用。

摘要

Organizing a large-scale knowledge graph into a typed property graph requires structural decisions—which entities become nodes, which properties become edges, and what schema governs these choices. Existing approaches embed these decisions in pipeline code (YAGO, DBpedia) or extract relations ad hoc (LLM-based pipelines), producing schemas that are tightly coupled to their construction process and difficult to reuse for downstream ontology-level tasks.

把一个大规模知识图谱组织成带类型的属性图，需要做出若干结构性决定：哪些实体成为节点、哪些属性成为边、由什么 schema 来管这些选择。现有做法要么把这些决定嵌进流水线代码（YAGO、DBpedia），要么临时抽取关系（各种基于大模型的流水线），产出的 schema 与其构建过程紧耦合，难以在下游的本体级任务中复用。

We present an ontology-oriented approach to knowledge graph construction: one in which the schema is designed from the outset for ontology analysis, entity disambiguation, domain customization, and LLM-guided extraction—not merely as a byproduct of graph building. The core mechanism is intrinsic-relational routing, which classifies every property as either intrinsic (a node attribute for lookup, e.g., birth date) or relational (a traversable graph edge, e.g., employer), and routes it to the corresponding schema module. This routing produces a declarative schema that is portable across storage backends and independently reusable.

我们提出一种本体导向的知识图谱构建方法：其中的 schema 从一开始就是为本体分析、实体消歧、领域定制和大模型引导的抽取而设计的，而不仅仅是构图的副产品。核心机制是内在-关系路由：它把每一个属性判成内在的（供查询用的节点属性，例如出生日期）或关系的（可遍历的图边，例如雇主），并路由到对应的 schema 模块。这一路由产出一份声明式 schema，可跨存储后端移植，并能被独立复用。

We instantiate the approach on the January 2026 Wikidata dump (about 100 M items). A rule-based cleaning stage identifies a 34.6 M-entity core set from the full dump, followed by iterative intrinsic-relational routing that assigns each property to one of 94 modules (56 intrinsic, 38 relational) organized into 8 categories. With tool-augmented LLM support and human review, the schema reaches 93.3% category coverage on the core set and 98.0% module assignment among classified entities. Exporting this schema yields a property graph with 34.0 M nodes and 61.2 M edges across 38 relationship types.

我们在 2026 年 1 月的 Wikidata 快照（约 1 亿条目）上实例化这套方法。一个基于规则的清洗阶段从全量快照中识别出 3460 万实体的核心集，随后用迭代式的内在-关系路由，把每个属性分配到 94 个模块之一（56 个内在、38 个关系），这些模块被组织进 8 个大类。在带工具的大模型支持与人工审阅之下，schema 在核心集上达到 93.3% 的大类覆盖率，在已分类实体中达到 98.0% 的模块分配率。导出该 schema 得到一个属性图，含 3400 万节点、6120 万条边，跨 38 种关系类型。

一、引言

Knowledge graphs have become a foundational structure for organizing information, from domain-specific applications to world-scale knowledge bases. Among these, Wikidata stands as one of the largest collaborative knowledge graphs, containing over 100 million entities. However, Wikidata's open editing model, while enabling broad coverage, comes at a cost: entity types are unconstrained, property usage is inconsistent, and no relationship schema is enforced. The result is a raw, semi-structured knowledge graph that is rich in content but difficult to query, analyze, or integrate into downstream applications at scale.

从领域特定的应用到世界级的知识库，知识图谱已成为组织信息的基础结构。其中，Wikidata 是最大的协作式知识图谱之一，包含超过 1 亿个实体。然而，Wikidata 的开放编辑模式虽然带来了广泛的覆盖，代价也是有的：实体类型没有约束，属性使用不一致，也没有强制的关系 schema。结果是一个原始的、半结构化的知识图谱——内容丰富，但难以大规模地查询、分析，或集成进下游应用。

Several projects have sought to impose structure on Wikidata. DBpedia and YAGO 4.5 extract and organize Wikidata content into cleaner ontological frameworks. However, these efforts focus on ontology alignment—mapping Wikidata into existing type hierarchies—rather than producing a schema that is itself designed for ontology-level applications such as structural analysis, entity disambiguation, or domain-specific subgraph extraction. Meanwhile, LLM-constructed knowledge graphs face the same disorder: recent pipelines extract entities and relations from unstructured text at scale, but the resulting graphs have ad hoc entity types, unconstrained relation vocabularies, and no shared schema governing what is extracted or how it is organized.

已有若干项目试图给 Wikidata 加上结构。DBpedia 和 YAGO 4.5 把 Wikidata 的内容抽取并组织进更干净的本体框架。然而这些努力聚焦于本体对齐——把 Wikidata 映射进已有的类型层级——而不是产出一份本身就为本体级应用（结构分析、实体消歧、领域子图抽取等）设计的 schema。与此同时，由大模型构建的知识图谱面临同样的无序：近期的流水线能大规模地从非结构化文本中抽取实体与关系，但得到的图有着临时拼凑的实体类型、没有约束的关系词汇表，也没有一份共同的 schema 来管「抽什么」和「怎么组织」。

What is missing is an ontology-oriented approach: one in which the schema is constructed with ontology applications in mind, so that the resulting artifact is not merely a side effect of graph building but a first-class resource for downstream tasks. Such a schema must be comprehensive in coverage, modular enough to support domain customization, and portable across storage backends—properties that existing pipeline-coupled schemas lack.

缺的是一种本体导向的方法：构建 schema 时就把本体类应用放在心上，使得产出的这份东西不只是构图的副作用，而是下游任务的一等资源。这样的 schema 必须覆盖全面、模块化程度足以支撑领域定制、并且能跨存储后端移植——而这些性质，现有那些与流水线绑死的 schema 都不具备。

Intrinsic modules route entity properties to typed tabular attributes suitable for node lookup, while relational modules route properties to graph edges that capture traversable connections.

Together, they define the node scope (which entities appear as nodes and what properties attach to them) and the edge boundary (which properties become traversable edges versus scalar attributes). The construction process preserves analyzable conceptual structure—explicit categories, reusable modules, principled property routing—while the produced artifact remains a declarative schema rather than a formal OWL/RDF ontology.

内在模块把实体属性路由成适合节点查询的带类型的表格字段，而关系模块把属性路由成图上的边，用来承载可遍历的连接。两者共同定义了「节点范围」（哪些实体作为节点出现、挂哪些属性）和「边界」（哪些属性成为可遍历的边、哪些只是标量属性）。构建过程保留了可分析的概念结构——显式的大类、可复用的模块、有原则的属性路由——而产出物仍然是一份声明式 schema，不是形式化的 OWL/RDF 本体。

二、相关工作

2.2 本体模块化与分面分类

The theoretical foundations for our two-layer framework draw from three traditions. Guizzardi's Unified Foundational Ontology (UFO) distinguishes rigid sortals (types essential to an entity's identity) from roles and mixins (contingent, relationally dependent types). Kiczales et al. identify an analogous pattern in software engineering: cross-cutting concerns that cannot be cleanly encapsulated in a single module of a system's primary decomposition. Ranganathan's faceted classification decomposes subjects into independent facets rather than forcing them into a single hierarchy.

我们这个两层框架的理论基础取自三个传统。Guizzardi 的统一基础本体（UFO）区分「刚性种类」（构成实体身份本质的类型）与「角色」和「混入」（偶然的、依赖关系而存在的类型）。Kiczales 等人在软件工程中识别出一个类似的模式：横切关注点，即那些无法被干净地封装进系统主分解结构中某一个模块的东西。Ranganathan 的分面分类法把主题拆解成若干互相独立的「面」，而不是硬把它们塞进一棵单一的层级树。

Our intrinsic-relational distinction operationalizes these ideas at scale: intrinsic modules behave like rigid sortals or primary facets (identity-providing, category-confined), while relational modules behave like mixins or cross-cutting concerns (spanning the primary category decomposition). A dominant heuristic—whether a module appears in multiple categories—guides this classification, reducing the degree of philosophical judgment that foundational ontology traditionally requires.

我们的内在-关系之分把这些想法在大规模上落地：内在模块的行为像刚性种类或主要的面（提供身份、局限在单个大类内），而关系模块的行为像混入或横切关注点（跨越主要的大类分解）。一条主导性的启发式规则——某个模块是否出现在多个大类中——指导这一分类，从而降低了基础本体传统上所要求的哲学判断的程度。

2.3 基于大模型的知识图谱构建

These approaches use LLMs to build schemas from text—discovering entity types and relations during extraction. Our work takes a complementary approach: we use a pre-built, iteratively refined schema to guide extraction, providing the LLM with a grounded type taxonomy and tag vocabulary rather than asking it to invent one.

那些做法是用大模型从文本中构建 schema——在抽取过程中发现实体类型与关系。我们的工作走的是互补的路径：我们用一份预先建好、迭代精化过的 schema 来引导抽取，给大模型一套有依据的类型分类法和标签词汇表，而不是让它自己去发明。

A further distinction is the role of the LLM itself. In existing pipelines, the LLM serves as an extractor—consuming text and producing graph triples. In our agentic oracle workflow, the LLM acts as a schema designer: it investigates unclassified entity types, proposes category and module assignments, and edits the declarative schema files. Grounding tools verify each identifier used in schema decisions against the knowledge graph at decision time, addressing the hallucination problem at its source rather than through post-hoc validation.

另一个区别在于大模型本身的角色。在已有的流水线里，大模型充当抽取器——吃进文本、产出图的三元组。而在我们的 agentic oracle 工作流里，大模型充当 schema 设计者：它调查那些未被分类的实体类型，提出大类与模块的归属建议，并直接编辑声明式 schema 文件。接地工具在做决定的当时，就把 schema 决策中用到的每一个标识符对着知识图谱核实一遍，从源头上解决幻觉问题，而不是靠事后校验。

三、方法

3.1 声明式分类 schema

The schema serves two purposes: it defines how to classify entities and what to extract from them. We represent it as a set of declarative configuration files—one per category—that jointly function as the single source of truth for both classification rules and output structure.

这份 schema 有两个用途：它定义「如何分类实体」和「从实体中抽取什么」。我们把它表示为一组声明式配置文件，每个大类一个文件，它们共同充当分类规则与输出结构的唯一事实来源。

Categories and gate matching. The schema partitions entities into k categories. Each category is defined by a set of gate values—entity types that trigger membership. An entity is assigned to a category if any of its type-assertion properties (e.g., instance-of or subclass-of) matches a value in that category's gate set. Categories are evaluated in a fixed priority order; the first match wins, ensuring mutual exclusivity. That is, the assigned categories form a partition over classified entities: each classified entity belongs to exactly one category.

大类与闸值匹配。这份 schema 把实体划分进 k 个大类。每个大类由一组「闸值」定义，即那些触发归属的实体类型。如果某个实体的任一类型断言属性（例如「是某类的实例」或「是某类的子类」）的取值命中了某个大类的闸值集合，该实体就被分到这个大类。各大类按固定的优先级顺序评估，先命中者胜，从而保证互斥。也就是说，所分配的大类在已分类实体上构成一个划分：每个已分类实体恰好属于一个大类。

Modules. Within each category, the schema defines a set of named modules that group semantically related properties. Each module has three fields: Type (intrinsic or relational): determines whether the module's properties are routed to node attributes (for lookup) or graph edges (for traversal). This is the edge boundary decision, made explicitly at schema design time. Indicators: a set of property-value conditions that trigger module assignment. When the value set is empty, the indicator is presence-based: the property exists on the entity, regardless of its values. Otherwise the indicator is value-based: the property must contain a value from the specified set. Value properties: the properties extracted for matched entities under this module.

模块。在每个大类内部，schema 定义一组有名字的模块，把语义相关的属性归到一起。每个模块有三个字段。类型（内在或关系）：决定这个模块的属性是被路由成节点属性（供查询）还是图上的边（供遍历）。这就是「边界」决定，它在 schema 设计时就被显式做出。指示器：一组属性-取值条件，命中则激活该模块。当取值集合为空时，这条指示器是「存在型」的——只要该属性在实体上存在即可，不看取值；否则是「取值型」的——该属性必须含有指定集合里的某个值。取值属性：对命中该模块的实体，要抽取哪些属性。

If a module is intrinsic, each extracted property-value pair becomes a node attribute; if it is relational and the value is itself an entity, it becomes a typed edge in the property graph. An entity receives one category, zero or more intrinsic modules, and zero or more relational modules. Intrinsic modules define what an entity is—identity-providing attributes that are typically category-confined (e.g., chemical formula for compounds, birth date for people). Relational modules define what an entity connects to—cross-cutting domains that often span multiple categories (e.g., award, military, religion).

如果一个模块是内在的，抽出的每个「属性-取值」对就成为一个节点属性；如果它是关系的、且取值本身是一个实体，那就成为属性图里一条带类型的边。一个实体拿到一个大类、零到多个内在模块、零到多个关系模块。内在模块定义一个实体「是什么」——那些提供身份、通常局限在单个大类内的属性（例如化合物的化学式、人的出生日期）。关系模块定义一个实体「连到什么」——那些常常跨越多个大类的横切领域（例如奖项、军事、宗教）。

The intrinsic - relational distinction is the default organizing principle; the schema designer may override it via the type assignment, for example reclassifying a module as relational when its values are entities worth traversing rather than scalar attributes.

内在-关系之分是默认的组织原则；schema 设计者可以通过类型指派来推翻它，例如当某个模块的取值是值得遍历的实体、而不是标量属性时，就把它改判为关系模块。

Illustrative example. Consider Apple Inc., whose instance-of values include enterprise and business. The gate set for organizations contains enterprise, so Apple is assigned to the organizations category. The first category match wins; Apple's remaining instance-of values do not trigger any other category. Once classified, module indicators are evaluated. The corporation module (intrinsic) activates because Apple's instance-of values match its value-based indicator set and it possesses the chief-executive-officer and employees properties. Its

value properties—employees, stock exchange—become node attributes for lookup. The affiliation module (relational) activates on properties like founded-by and child-organization. Because affiliation is relational and the values are entities, each becomes a typed edge. In total, Apple receives 1 intrinsic module and 8 relational modules.

举例说明。以苹果公司为例，它的「是某类的实例」取值包含「企业」和「商业机构」。organizations（组织）这个大类的闸值集合里含有「企业」，因此苹果被分到 organizations 大类。先命中者胜，苹果剩下的实例取值不再触发其他大类。分好类之后，开始评估模块的指示器。corporation（公司）这个内在模块被激活，因为苹果的实例取值命中了它的取值型指示器集合，并且苹果拥有「首席执行官」和「员工数」这两个属性。该模块的取值属性——员工数、上市交易所——成为供查询的节点属性。affiliation（关联）这个关系模块被「由谁创立」「下属组织」这类属性激活。由于 affiliation 是关系模块、取值又是实体，每一个取值都成为一条带类型的边。总计，苹果拿到 1 个内在模块和 8 个关系模块。

3.2 迭代式 schema 精化

The schema is not designed in a single pass. It emerges through an iterative refinement loop that alternates between classification and failure analysis. The process begins with a seed schema—a small set of gate values, a handful of modules per category, and an initial list of value properties—that reflects the builder's intent for the target graph. Each round identifies two failure sets: unclassified entities (no gate match) and no-module entities (classified but no module activated), then applies three decision oracles to resolve them.

这份 schema 不是一次设计成的。它是在「分类」与「失败分析」交替的迭代精化循环中长出来的。流程从一个种子 schema 开始——一小组闸值、每个大类少量模块、一份初始的取值属性清单——它体现了构建者对目标图的意图。每一轮识别两个失败集合：未分类实体（没有闸值命中）和无模块实体（已归类但没有任何模块被激活），然后用三个决策 oracle 去解决它们。

The computational steps—classification, failure partitioning, rate computation—are deterministic and parallelizable. The bottleneck lies in three decision oracles that require semantic judgment: (1) Category oracle: given an unclassified type value, assign it to a category. This drives the gate expansion step. (2) Module oracle: given a new gate value with no matching indicator, assign it to a module within its category. (3) Refinement oracle: given the current schema and failure analysis, decide whether to create, merge, or split modules, and set their parameters.

计算步骤（分类、失败集划分、比率计算）是确定性且可并行的。瓶颈在于三个需要语义判断的决策 oracle：其一，大类 oracle：给定一个未被分类的类型取值，把它分给某个大类，这一步驱动闸值扩张；其二，模块 oracle：给定一个新的闸值、且没有任何指示器与之匹配时，把它分给该大类内的某个模块；其三，精化 oracle：给定当前 schema 与失败分析，决定要不要新建、合并或拆分模块，并设定它们的参数。

Traditionally, these oracles require expensive human annotation or purpose-built classifiers trained on labeled data. We describe an alternative: an agentic LLM workflow equipped with

grounding tools (entity queries, property lookups, sample inspection) that implements all three oracles, reducing both human effort and hallucination risk.

传统上，这些 oracle 需要昂贵的人工标注，或者需要在标注数据上训练的专用分类器。我们给出一个替代方案：一个配备了接地工具（实体查询、属性查询、样本检视）的 agentic 大模型工作流，由它实现这三个 oracle，从而同时降低人力成本和幻觉风险。

Convergence. The schema evolves through additive updates to gates and indicators, with periodic module restructuring during refinement. Under fixed category priority, gate expansion typically improves category coverage. By contrast, the module rate is not guaranteed to be monotonic: reassignment and module updates can temporarily increase or decrease the no-module set before later rounds recover coverage. The loop terminates when both rates reach their targets or when no further oracle assignments are made. In practice, we set the thresholds to about 0.9, since a long tail of rare or ambiguous types may not belong in any category; the oracles correctly decline to assign them, and the remaining gap reflects genuine coverage limits of the schema rather than algorithmic failure.

收敛性。schema 通过对阈值与指示器做增量更新而演化，并在精化过程中周期性地重构模块。在固定的大类优先级下，阈值扩张通常会提升大类覆盖率。相比之下，模块分配率不保证单调：重新指派和模块更新可能会先让无模块集合变大或变小，之后的轮次才把覆盖补回来。当两个比率都达到目标、或者 oracle 不再做出新的指派时，循环终止。实践中我们把两个阈值都设在 0.9 左右，因为稀有或含糊的类型有一条长尾，它们本来就可能不属于任何大类；oracle 正确地拒绝为它们分配，剩下的差距反映的是 schema 覆盖的真实边界，而不是算法的失败。

四、Wikidata 案例研究

4.1 数据清洗

The first step establishes graph scope: which entities belong in the graph at all. Wikidata's 100 M entities include a substantial proportion of automated bulk imports and infrastructure entities that are not suitable for an encyclopedic knowledge graph. Data cleaning separates these from genuine editorial content through a priority-ordered cascade: (1) Structural classification: detection using 45 infrastructure identifiers (e.g., Wikimedia disambiguation pages, categories) and 28 bulk-import identifiers (e.g., scientific article, gene). (2) Source signature matching: 19 known import databases (e.g., CrossRef, astronomy catalogs, taxonomy databases) are detected via characteristic property combinations. (3) Curation score: a weighted curation score protects notable entities from false positives—an entity with rich editorial signals is retained even if its type matches a bulk-import pattern. (4) Ratio-based safety net: remaining entity types with a bulk-import property ratio of 70% or more and insufficient curation signals are automatically classified as non-core.

第一步是确定「图的范围」：哪些实体压根就该进这个图。Wikidata 的 1 亿实体里，有相当大比例是自动批量导入的条目和基础设施条目，它们不适合放进一个百科式的知识图谱。数据清洗通过一个按优先级排列的级联流程，把这些东西与真正的人工编辑内容分开：其一，结构性分类，用 45 个基础设施标识符（例如维基百科歧页、分类页）和 28 个批量导入标识符（例如科学论文、基因）来检出；其二，来源特征匹配，通过特征性的属性组合识别出 19 个已知的导入数据库（例如 CrossRef、天文星表、生物分类数据库）；其三，策展分数，一个加权的策展分数保护知名实体不被误杀——一个实体只要有丰富的编辑信号，即使它的类型命中了批量导入的模式，也会被保留；其四，基于比率的安全网，剩下那些「批量导入属性占比达到或超过 70%、且策展信号不足」的实体类型，被自动判为非核心。

（原文脚注给出策展分数的构成：36 个属性分五档，权重 1 到 3。强信号如图片、坐标、官网计 3 分；中等信号如奖项、代表作计 2 分；弱信号如国别、行业计 1 分；对人物另设专门档位，如职业、教育背景。）

The cleaning stage is rule-based and deterministic given fixed rule definitions: every tier assignment traces to a specific match. Prior work on Wikidata quality focuses on statement-level accuracy and completeness; our cleaning addresses a complementary dimension—entity-level provenance—and is designed to separate likely human-curated content from likely automated bulk imports. After cleaning, 34.6 M entities remain as the core dataset for classification.

在规则定义固定的前提下，清洗阶段是基于规则的、确定性的：每一次档位指派都能追溯到某一条具体的匹配。以往关于 Wikidata 质量的工作聚焦于陈述级的准确性与完整性；我们的清洗处理的是一个互补的维度——实体级的来源出处——目的是把可能由人工策展的内容与可能是自动批量导入的内容分开。清洗之后，剩下 3460 万实体作为分类用的核心数据集。

4.2 schema 实例化与精化

The seed schema was constructed by incorporating three sources: Schema.org's type hierarchy for broad category structure, YAGO 4.5's curated taxonomy for entity type coverage, and Wikidata's own EntitySchema definitions for property-level detail. From these, we derived eight categories: people, places, creative_works_media, knowledge, science, organizations, events_actions, and products_artifacts. The schema is defined in eight YAML files, each specifying gate identifiers, core properties, and module definitions with indicators and value properties.

种子 schema 由三个来源组合而成：Schema.org 的类型层级提供粗粒度的大类结构；YAGO 4.5 的策展分类法提供实体类型的覆盖；Wikidata 自己的 EntitySchema 定义提供属性级的细节。由此我们得出八大类：人物、地点、创意作品与媒体、知识、科学、组织、事件与行动、产品与人造物。schema 定义在八个 YAML 文件里，每个文件写明阈值标识符、核心属性，以及带指示器和取值属性的模块定义。

Refinement process. The Rust classifier processes the full 34.6 M entity set in approximately five minutes (about 110,000 entities per second), making A/B comparison practical for every schema edit. Final schema: after convergence, the refined schema achieves a classification rate

of 93.3% (32.3 M of 34.6 M core entities) and a module-assignment rate of 98.0% among classified entities (no-module rate reduced from 15.8% to 2.0%). The schema comprises 94 modules: 56 intrinsic and 38 relational. Intrinsic modules are category-confined (e.g., film in creative_works_media, river in places). Relational modules capture cross-cutting domains: 18 span three or more categories (e.g., military in 6, religion in 7).

精化过程。用 Rust 写的分类器处理全部 3460 万实体大约需要五分钟（约每秒 11 万条），使得每一次改动 schema 都能做 A/B 对比。最终 schema：收敛之后，精化后的 schema 达到 93.3% 的分类率（3460 万核心实体中的 3230 万），以及在已分类实体中 98.0% 的模块分配率（无模块比例从 15.8% 降到 2.0%）。该 schema 含 94 个模块：56 个内在、38 个关系。内在模块局限在单个大类内（例如「电影」在创意作品与媒体里，「河流」在地点里）。关系模块承载横切领域：其中 18 个横跨三个或更多大类（例如军事跨 6 个，宗教跨 7 个）。

4.3 agentic oracle 的实现

The three decision oracles are implemented as an agentic workflow powered by Claude Opus 4.6. Two challenges motivate the tool design. First, LLMs hallucinate Wikidata identifiers: given a label, an LLM may fabricate a plausible but nonexistent identifier, and given an identifier, it may invent an incorrect label. Second, the agent cannot reliably classify an unfamiliar entity type from its label alone—it requires seeing sample entities, instance counts, and descriptions to make an informed category decision.

三个决策 oracle 实现为一个由 Claude Opus 4.6 驱动的 agentic 工作流。有两个挑战决定了工具的设计。第一，大模型会对 Wikidata 的标识符产生幻觉：给它一个名字，它可能编造出一个看起来合理但并不存在的标识符；给它一个标识符，它可能编出一个错误的名字。第二，agent 单凭一个名字无法可靠地给一个陌生的实体类型分类——它需要看到样本实体、实例数量和描述，才能做出有依据的大类判断。

To address both challenges, the agent is equipped with grounding tools that provide verified information from the knowledge graph: a label lookup that resolves identifiers to their canonical English labels and vice versa, against a memory-mapped store built once from a full scan of the dump—every identifier the agent references is verified through this tool; a live type query that retrieves entity counts, labels, descriptions, and sample instances for a given type value; a tag validator that validates YAML schema files for structural correctness, gate - module synchronization, identifier format, and cross-category gate exclusivity; a category analysis tool that produces per-category coverage statistics, frequency tables, cross-tabulations, and reports of entities with no module assignment—diagnosing exactly where the schema fails; and an unclassified hub analysis that identifies high-reference unclassified entities and ranks them by inbound reference count—surfacing the entity types with the greatest impact on graph connectivity.

为应对这两个挑战，agent 被配上了一批「接地工具」，用于从知识图谱提供经过核实的信息：标签查询工具，把标识符解析成规范英文名（反向也行），查询对象是一个只做一次全量扫描就建好的内存映射存储——

agent 引用的每一个标识符都要经这个工具核实；在线类型查询工具，对给定的类型取值取回实体数量、名字、描述和样本实例；标签校验器，校验 YAML schema 文件的结构正确性、阈值与模块的同步、标识符格式，以及跨大类的阈值互斥性；大类分析工具，产出每个大类的覆盖统计、频次表、交叉表，以及那些没有分到任何模块的实体报告——精确诊断 schema 在哪里失效；未分类枢纽分析工具，找出那些被大量引用却仍未分类的实体，并按入向引用数排序——把对图连通性影响最大的实体类型浮出水面。

A key design choice in the YAML schema is that every identifier must carry an inline comment with its canonical English label, verified against the label store by the tag validator. This makes the schema files self-documenting: the agent can review and edit definitions by reading the YAML directly, without querying each identifier individually.

YAML schema 里的一个关键设计选择是：每一个标识符都必须带一条行内注释，写上它的规范英文名，并由标签校验器对着标签库核实。这让 schema 文件成为自文档化的：agent 可以直接读 YAML 来审阅和编辑定义，不必逐个去查每个标识符。

Investigation phase. The agent examines two priority lists: the most frequent unclassified entity types (for coverage) and the highest-reference hub types (for graph connectivity—these may be infrequent but heavily referenced by classified entities). For each candidate, the agent queries labels and sample entities via the grounding tools, then generates a report recommending which category, which module, and whether new modules are needed. This report is presented for human review; the reviewer may accept, reject, or annotate individual recommendations.

Implementation phase. The agent applies the approved changes to the YAML files, using the label lookup to ensure correct annotations and the tag validator to check structural integrity. The human operator then triggers reclassification to measure the coverage improvement, closing the loop for the next refinement round.

调查阶段。agent 检视两份优先清单：出现频次最高的未分类实体类型（为了覆盖率），以及入向引用最多的枢纽类型（为了图的连通性——这些类型可能出现频次不高，但被大量已分类实体引用）。对每个候选，agent 通过接地工具查询名字和样本实体，然后生成一份报告，建议归入哪个大类、哪个模块，以及是否需要新建模块。这份报告提交人工审阅；审阅者可以逐条接受、驳回或加注。实施阶段。agent 把获批的改动落到 YAML 文件里，用标签查询确保注释正确，用标签校验器检查结构完整性。随后由人工操作者触发重新分类，测量覆盖率的提升，为下一轮精化闭环。

4.4 分类产物

The primary output is a set of Parquet files (13 columns per entity) containing category, module assignments, and claims partitioned into four buckets: core (shared identity attributes), intrinsic (category-specific scalar properties), relational (entity-to-entity connections), and unclaimed (remaining properties). Each claim carries its property label, resolved value label, qualifiers, and a pre-generated natural-language sentence (e.g., "James Humphrey Walwyn's occupation is military officer (military branch: Royal Navy).") with all identifiers replaced by their English labels. These self-contained files serve as the single source for all downstream outputs: the core and

intrinsic buckets provide lookup-table information for each node, the relational bucket drives graph edge construction, and the sentence field enables direct use in LLM prompts or text-based retrieval without additional ID resolution.

主要产物是一组 Parquet 文件（每个实体 13 列），包含大类、模块归属，以及被划进四个桶的声明：core（共享的身份属性）、intrinsic（大类专属的标量属性）、relational（实体到实体的连接）、unclaimed（剩下的属性）。每条声明都带有它的属性名、已解析的取值名、限定词，以及一句[预先生成好的自然语言句子](#)（例如「James Humphrey Walwyn 的职业是军官（军种：皇家海军）。」），句中所有标识符都已替换成英文名。这些自包含的文件充当所有下游产物的唯一来源：core 桶和 intrinsic 桶为每个节点提供查表信息，relational 桶驱动图的边构建，而那个句子字段使得这些内容能直接用在大模型的 prompt 里、或者用于基于文本的检索，无需再做标识符解析。

For property graph database import, a three-pass export pipeline produces graph-ready CSVs: node export (one file per category, entity identifier, label, description, and core properties as node attributes, with intrinsic module properties as additional columns); edge export (one file per relational module, source, target, relationship type, and qualifier properties—only value properties whose targets are themselves entities produce edges); and stub node export (edge targets not in the core dataset are exported as stub nodes with identifier and label only, preserving graph connectivity without inflating the classified node set).

为了导入属性图数据库，一个三趟导出流水线产出可直接入库的 CSV：节点导出（每个大类一个文件，包含实体标识符、名字、描述，以及作为节点属性的核心属性，内在模块的属性作为额外列）；边导出（每个关系模块一个文件，包含源、目标、关系类型和限定词属性——只有目标本身是实体的取值属性才产生边）；桩节点导出（那些不在核心数据集中的边目标，导出为只有标识符和名字的桩节点，既保住图的连通性，又不让已分类节点集虚胖）。

This maps directly to the intrinsic - relational distinction: intrinsic modules become node attributes (suitable for tabular stores or property lookups), while relational modules become typed edges (suitable for graph traversal). The separation ensures that each storage engine carries only the data matching its primary access pattern. The final graph contains 32.3 M classified entity nodes, 1.7 M stub nodes, and 61.2 M edges across 38 relationship types.

这直接对应内在-关系之分：内在模块成为节点属性（适合表格存储或属性查询），关系模块成为带类型的边（适合图遍历）。这一分离确保每个存储引擎只承载与其主要访问模式相匹配的数据。最终的图包含 3230 万个已分类实体节点、170 万个桩节点，以及跨 38 种关系类型的 6120 万条边。

五、应用

5.1 本体结构

The bipartite structure reveals two patterns. First, relational modules vary widely in span: religion connects 7 categories, while authorship and politics each bridge 3. Second, the modules form

natural domain clusters: governance-related modules (government, legal, politics) share the same category connections, as do cultural modules (culture, entertainment, sports, religion) and economic modules (finance, food, transportation). These clusters suggest that the relational modules capture coherent thematic domains rather than arbitrary property groupings.

这个二分图结构揭示了两个模式。第一，关系模块的跨度差异很大：宗教连接 7 个大类，而著作权和政治各只桥接 3 个。第二，这些模块自然形成领域簇：治理类模块（政府、法律、政治）共享相同的大类连接，文化类模块（文化、娱乐、体育、宗教）和经济类模块（金融、食品、交通）也各自成簇。这些簇说明关系模块捕捉到的是融贯的主题领域，而不是随意的属性分组。

These domain clusters directly enable topic-centered subgraph extraction. Selecting a cluster of relational modules produces a self-contained graph with its own node types, edge types, and schema. Selecting government, legal, and politics yields a geopolitical subgraph; healthcare plus education plus Science yields a biomedical research subgraph; and culture plus entertainment plus sports yields a cultural analysis subgraph—each derived entirely from module selection.

这些领域簇直接支撑「以主题为中心的子图抽取」。挑一簇关系模块，就能得到一个自包含的图，带有自己的节点类型、边类型和 schema。挑政府、法律、政治，得到一个地缘政治子图；挑医疗、教育加科学，得到一个生物医学研究子图；挑文化、娱乐、体育，得到一个文化分析子图——每一个都完全由「选哪些模块」推导出来。

Beyond schema-level analysis, the classification system also operates at the individual entity level, transforming an unstructured bag of Wikidata properties into an organized view. A live query for Apple Inc. currently carries 248 raw properties with no grouping beyond property IDs. The classifier routes it to organizations and assigns properties to structured groups.

除了 schema 层面的分析，这套分类系统也在单个实体的层面起作用：把一堆无结构的 Wikidata 属性变成一个有组织的视图。对苹果公司做一次实时查询，目前会拿到 248 条原始属性，除了属性编号之外没有任何分组。分类器把它路由到「组织」大类，并把属性分派进结构化的分组。

5.3 实体画像与消歧

The module system produces structured entity profiles that serve as discriminative type annotations for entity disambiguation. We evaluate this on the BLINK benchmark, comparing our module-based type annotations against YAGO 4.5's curated type hierarchy. To isolate the effect of type annotations from candidate retrieval, we compare on the 12,323 mentions where both systems return the same candidate entities.

这套模块系统产出结构化的实体画像，可以充当实体消歧时有区分力的类型标注。我们在 BLINK 评测集上做了评估，把我们基于模块的类型标注与 YAGO 4.5 的策展类型层级做对比。为了把「类型标注的效果」与「候选检索的效果」隔离开，我们只在两个系统返回相同候选实体的 12323 条指称上做比较。

The module-based approach achieves a 2.4 percentage point improvement in macro average accuracy (83.9% vs. 81.5%). Gains are largest in the hardest group (20+ candidates, +8.0 points), where richer type annotations provide the most disambiguation signal; because this bucket contains only 25 mentions, the effect size should be interpreted as directional. The improvement is driven by higher annotation density: module-based candidates carry an average of 2.52 type labels per entity compared to YAGO's 1.42.

基于模块的做法在宏平均准确率上提升 2.4 个百分点（83.9% 对 81.5%）。增益在最难的那一档最大（候选 20 个以上，高 8.0 个百分点），因为在那里更丰富的类型标注提供了最多的消歧信号；但由于这一档只含 25 条指称，[这个效应量应当被理解为方向性的](#)。提升的驱动因素是标注密度更高：基于模块的候选平均每个实体带 2.52 个类型标签，而 YAGO 是 1.42 个。

Three mechanisms contribute: (1) relational modules extract fine-grained attributes (e.g., the sports module routes to playing position, distinguishing "running back" from "defensive end" where YAGO assigns identical "American Football Player" types); (2) the module system recovers specific occupations for entities YAGO labels only as generic "Person" (30.8% of YAGO's exclusive failures); and (3) intrinsic modules provide domain signals that further distinguish entities sharing identical instance-of types.

有三个机制在起作用：其一，关系模块抽出细粒度属性（例如体育模块会路由到「场上位置」，从而区分「跑锋」和「防守端锋」，而 YAGO 对这两者赋予相同的「美式橄榄球运动员」类型）；其二，对那些 YAGO 只标成笼统「人」的实体，模块系统能找回具体职业（占 YAGO 独有失败案例的 30.8%）；其三，内在模块提供领域信号，进一步区分那些「是某类的实例」取值完全相同的实体。

5.4 schema 定制

The declarative YAML schema is designed as a parameterized schema generator: the generic definitions (8 categories, 94 modules) serve as a template that domain experts can customize without code changes. Customization follows three operations: selecting which categories and modules to include, decomposing broad modules into fine-grained facets, and adding domain-specific gate identifiers or indicators. The entire pipeline adapts to whatever YAML definitions are provided.

这份声明式 YAML schema 被设计成一个参数化的 schema 生成器：通用定义（8 个大类、94 个模块）充当模板，领域专家可以在不改代码的情况下定制它。定制有三种操作：[选择](#)要纳入哪些大类和模块；把宽泛的模块[分解](#)成细粒度的面；[添加](#)领域专属的阈值标识符或指示器。整条流水线会适配所给的任何一份 YAML 定义。

We demonstrate this with the education module in people, which contains approximately 90 occupation indicators spanning natural sciences, engineering, humanities, social sciences, and teaching. In the generic schema, a marine biologist and a medieval historian both receive the same education tag with no further distinction. By decomposing this single module into 9

domain facets—e.g., natural_science, humanities, engineering—users obtain fine-grained classification while preserving the same YAML structure. Only the indicator values change; the pipeline works unchanged.

我们以「人物」大类下的 education（教育）模块为例演示：它含约 90 个职业指示器，横跨自然科学、工程、人文、社会科学和教学。在通用 schema 里，一个海洋生物学家和一个中世纪史学家都拿到同一个 education 标签，再无区分。把这一个模块分解成 9 个领域面（例如自然科学、人文、工程）之后，用户就得到细粒度的分类，而 YAML 结构保持不变。**改的只是指示器的取值；流水线一行不用动。**

5.5 大模型引导的实体抽取

The YAML schema serves as a lightweight ontological structure for LLM-based entity extraction. The 8 categories define an entity taxonomy, while module names act as fine-grained semantic tags. An LLM can use this structure to extract entities from unstructured text and classify them with contextually relevant facets—without fine-tuning. A prompt template is constructed directly from the schema: categories provide the entity type taxonomy, and each category's module names provide the tag vocabulary. The LLM receives a text passage and returns structured output—entity name, category, and context-relevant tags.

这份 YAML schema 充当基于大模型的实体抽取所用的轻量本体结构。8 个大类定义了实体分类法，而模块名充当细粒度的语义标签。大模型能用这个结构从非结构化文本中抽取实体，并给它们打上与上下文相关的面——**不需要微调**。prompt 模板直接从 schema 构造出来：大类提供实体类型分类法，每个大类的模块名提供标签词汇表。大模型接收一段文本，返回结构化输出——实体名、大类、与上下文相关的标签。

（原文表 4 给出对一段关于水星牌汽车的维基百科文本的抽取结果，模型用 GPT-5-mini：实体「Mercury」，大类 Product，标签 brand/vehicle/transportation；实体「Ford Motor Company」，大类 Organization，标签 corporation/transportation；实体「Edsel Ford」，大类 Person，标签 creativity；实体「Lincoln-Mercury Division」，大类 Organization，标签 affiliation/corporation。）

This approach has two advantages over ad-hoc extraction schemas. First, the tag vocabulary is grounded in a schema that has been iteratively refined against 32.3 M classified entities, supporting broad coverage with reduced redundancy in practice. Second, extracted entities can be directly linked to the knowledge graph via category and module alignment, enabling seamless integration of LLM-extracted information with the existing graph structure.

相比临时拼凑的抽取 schema，这个做法有两个优势。第一，标签词汇表有依据——它是在 3230 万已分类实体上迭代精化出来的，实践中既覆盖广又减少了冗余。第二，抽出来的实体可以通过大类与模块的对齐，直接链接到知识图谱上，从而让大模型抽取的信息与已有图结构无缝集成。

六、结论

We presented an ontology-oriented approach to knowledge graph construction grounded in a single design principle: routing each property as either intrinsic—stored as a node attribute for

lookup—or relational—exported as a traversable edge. Formalizing this distinction into a declarative schema brings two benefits. First, it makes structural decisions explicit and inspectable, replacing ad-hoc property handling embedded in construction code. Second, it decouples the schema from the pipeline that produced it, enabling independent reuse for downstream tasks.

我们提出了一种本体导向的知识图谱构建方法，它建立在一条单一的设计原则上：把每一个属性路由成内在的（作为供查询的节点属性存储）或关系的（导出为可遍历的边）。把这一区分形式化成一份声明式 schema 带来两个好处。第一，它让结构性决定变得显式、可检视，取代了埋在构建代码里的临时属性处理。第二，它把 schema 与产出它的流水线解耦，使其能在下游任务中被独立复用。

Several directions remain open. The current schema was refined against the January 2026 Wikidata dump, but graph expansion from external sources such as LLM-extracted knowledge graphs will introduce relation types that do not align cleanly with the existing module inventory. Accommodating such expansion requires schema-extension methods that can map new relations to existing modules when possible, extend modules when needed, and introduce new modules when no faithful alignment exists. A systematic study of schema-guided retrieval—using the intrinsic-relational structure to shape context selection for retrieval-augmented generation—is a natural next step.

还有若干方向没有解决。当前的 schema 是针对 2026 年 1 月的 Wikidata 快照精化出来的，但从外部来源（例如由大模型抽取出的知识图谱）扩展图时，会引入与现有模块清单对不齐的关系类型。容纳这种扩展需要一套 schema 扩展方法：能映射到现有模块时就映射，需要时扩展模块，实在没有忠实对齐时就引入新模块。另一个自然的下一步是系统研究「schema 引导的检索」——用内在-关系结构来塑造检索增强生成时的上下文选择。

参考链接

- [OntoKG 代码与 schema](#) — 论文开源的 YAML schema 与两套实现（Rust 与 Python）
- [论文原文 arXiv:2604.02618](#) — 本文原始出处
- [YAGO 4.5](#) — 论文的主要对照系统，基于 schema.org 词汇表构建的 Wikidata 类型化知识库
- [Wikidata](#) — 论文的数据来源，1 亿条目的协作式知识库